

ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ
ΣΧΟΛΗ ΗΛΕΚΤΡΟΛΟΓΩΝ ΜΗΧΑΝΙΚΩΝ & ΜΗΧΑΝΙΚΩΝ ΥΠΟΛΟΓΙΣΤΩΝ



Chordify: A Chord DHT implementation

ΕΞΑΜΗΝΙΑΙΑ ΕΡΓΑΣΙΑ ΣΤΑ ΚΑΤΑΝΕΜΗΜΕΝΑ ΣΥΣΤΗΜΑΤΑ
6 ΜΑΡΤΙΟΥ 2026

ΣΚΟΤΗΣ ΑΠΟΣΤΟΛΟΣ
ΑΜ: 03121212

ΡΕΪΖΗΣ ΕΜΜΑΝΟΥΗΛ
ΑΜ: 03121067

ΔΗΜΗΤΡΑΚΟΠΟΥΛΟΣ ΔΗΜΗΤΡΗΣ
ΑΜ: 03121066

1 Εισαγωγή

1.1 Παρουσίαση Εφαρμογής

Στα πλαίσια της παρούσας εργασίας, σχεδιάστηκε και υλοποιήθηκε το Chordify, μια Peer-to-Peer εφαρμογή ανταλλαγής τραγουδιών, η οποία βασίζεται στην αρχιτεκτονική Chord DHT. Σκοπός του συστήματος είναι να λειτουργεί ως ένα distributed key-value storage, όπου οι χρήστες συνδέονται, μοιράζονται τα τραγούδια που διαθέτουν και πραγματοποιούν αναζητήσεις με βάση τον τίτλο του τραγουδιού. Ο κώδικας της εφαρμογής μας βρίσκεται στο ακόλουθο [repo](#).

Στη συγκεκριμένη υλοποίηση, το key αντιστοιχεί στο όνομα του τραγουδιού, ενώ η τιμή value είναι μια τυχαία συμβολοσειρά (σε μια πραγματική εφαρμογή θα ήταν ένα αρχείο ήχου π.χ. mp3). Η εφαρμογή εστιάζει αποκλειστικά στον εντοπισμό του αρχείου μέσω του δικτύου και όχι στην ίδια τη μεταφόρτωση των αρχείων. Το ID κάθε κόμβου στο δακτύλιο προκύπτει από την εφαρμογή της συνάρτησης κατακερματισμού SHA-1 στον συνδυασμό ip_address:port του κόμβου.

Οι κόμβοι επικοινωνούν πλήρως ασύγχρονα και λειτουργούν ως multithreaded servers, αποθηκεύοντας τα δεδομένα τους σε HashMaps με χρήση μηχανισμών reader-writer locks για ασφαλή ταυτόχρονη προσπέλαση. Το σύστημα υποστηρίζει πλήρως τη δρομολόγηση δακτυλίου καθώς και τις βασικές λειτουργίες διαχείρισης κόμβων και δεδομένων: **Join**, **Depart**, **Insert**, **Delete** και **Query**.

Τέλος, για την εξασφάλιση της διαθεσιμότητας των δεδομένων, υλοποιήθηκε μηχανισμός replication. Προκειμένου να μελετηθεί η συμπεριφορά του συστήματος, αναπτύχθηκαν και συγκρίθηκαν δύο διαφορετικά μοντέλα συνέπειας: το Chain Replication για ισχυρή συνέπεια (linearizability) και το Eventual Consistency για χαλαρή συνέπεια με τελική σύγκλιση.

1.2 Τεχνικές Λεπτομέρειες

Για την ανάπτυξη του συστήματος, αν και η επιλογή γλώσσας ήταν ελεύθερη, επιλέχθηκε η γλώσσα προγραμματισμού **Rust**. Η συγκεκριμένη επιλογή έγινε με κύριο γνώμονα την υψηλή απόδοση και την ασφάλεια που προσφέρει, ειδικά σε απαιτητικά περιβάλλοντα ταυτόχρονης εκτέλεσης. Δεδομένου ότι ο κάθε κόμβος λειτουργεί ως multithreaded server και απαιτείται ταυτόχρονη, ασφαλής προσπέλαση σε κοινούς πόρους (HashMaps), το αυστηρό σύστημα τύπων και ο borrow checker της Rust εγγυώνται την απουσία data races και την ασφαλή διαχείριση της μνήμης. Αυτά τα χαρακτηριστικά επιτρέπουν στο Chordify να εκτελείται αποδοτικά, επιτυγχάνοντας βέλτιστο throughput και αξιοπιστία κατά την εκτέλεση των πειραμάτων.

Για τα πειράματα αναπτύχθηκαν 3 ξεχωριστά scripts για να αυτοματοποιηθεί η εκτέλεση τους και για τις τρεις περιπτώσεις. Συγκεκριμένα, τρέξαμε τα insert, query και request πειράματα και για τους δύο τύπους consistency για αριθμό αντιγράφων 1, 3, και 5. Ο κώδικας για τα scripts βρίσκεται στον chordify/src/bin και τα αποτελέσματα με τον σχολιασμό παρουσιάζονται ακολούθως.

2 Πειράματα

2.1 Πείραμα 1: Write Throughput vs. Replication Factor

Για το πρώτο πείραμα εισάγουμε σε έναν δακτύλιο δέκα κόμβους και κάθε ένας πραγματοποιεί από 50 εισαγωγές τραγουδιών. Τα δεδομένα εισόδου μας δίνονται έτοιμα σε αρχεία της μορφής `insert_0x_part.txt` όπου $x=0,1,\dots,9$. Η εισαγωγές γίνονται παράλληλα και από τους 10 κόμβους χρησιμοποιώντας shell scripts που βρίσκονται στο directory `experiments` (απαιτείται εγκατάσταση του `gnome-terminal`), ενώ ο κώδικας που εκτελεί ο κάθε κόμβος βρίσκεται στο αρχείο `insert_throughput.rs` στο directory `src/bin`. Οι κόμβοι καταγράφουν τον χρόνο που χρειάστηκαν οι εισαγωγές και τον τυπώνουν. Επειδή υπάρχουν αποκλίσεις στις υπολογιζόμενες τιμές σε κάθε `peer`, υπολογίζουμε τον μέσο όρο αυτών των 10 χρόνων ώστε να πάρουμε μία αντιπροσωπευτική τιμή. Στην συνέχεια, ο υπολογισμός της ρυθμαπόδοσης γίνεται διαιρώντας τον συνολικό αριθμό εισαγωγών δια τον χρόνο που υπολογίσαμε προηγουμένως.

Σε αυτό το σημείο να τονίσουμε ότι όλα τα πειράματα εκτελέστηκαν σε τοπικό μηχάνημα, χωρίς δηλαδή οι κόμβοι να γίνουν `deployed` σε πραγματικούς ξεχωριστούς υπολογιστές και να λάβουν πραγματικές `ip`. Η επικοινωνία πραγματοποιείται εξ ολοκλήρου με `localhost`, επομένως οι μετρήσεις που λαμβάνουμε δεν είναι πλήρως αντιπροσωπευτικές ενός πραγματικού συστήματος DHT και ενδέχεται να περιέχουν σφάλμα. Επιπλέον, σε αρκετές περιπτώσεις εμφανίζεται αρκετά μεγάλη απόκλιση τόσο μεταξύ των χρόνων κόμβων όσο και μεταξύ διαφορετικών επαναλήψεων των ίδιων πειραμάτων. Η χρήση του μέσου όρου αποσκοπεί στο να ελαττώσει αυτό το φαινόμενο. Ο επόμενος πίνακας περιλαμβάνει τα αποτελέσματα για τις δύο μορφές συνέπειας και για replication factor 1,3 και 5:

	k=1	k=3	k=5
t=0	2624.3	1915.1	1305.2
t=1	3571.4	3448.2	3086.4

Table 1: Ρυθμαπόδοση εγγραφών για διαφορετικούς συνδυασμούς συνέπειας και replication factor (πράξεις ανά δευτερόλεπτο)

Παρατηρούμε ότι γενικά, η αύξηση του replication factor περιορίζει τις επιδόσεις του συστήματος. Αυτό είναι λογικό δεδομένου ότι με περισσότερα αντίγραφα απαιτείται περισσότερη επικοινωνία ώστε αυτά να συγχρονιστούν. Αυτό ισχύει πιο έντονα για την περίπτωση του `linearizability`, στο οποίο, σύμφωνα με την `chain replication` υλοποίηση που ακολουθήσαμε, η εγγραφή πρέπει να διαδοθεί και στους $k-1$ επόμενους κόμβους προτού θεωρηθεί επιτυχής. Το `eventual consistency` σχήμα πρέπει και αυτό να προωθήσει την εισαγωγή στις replicas αλλά δεν χρειάζεται να περιμένει μέχρι αυτή η διαδικασία να ολοκληρωθεί. Έτσι, έχει βελτιωμένο throughput σε σχέση με το `eventual consistency`.

2.2 Πείραμα 2: Read Throughput vs. Replication Factor

Στο δεύτερο πείραμα θα πραγματοποιήσουμε αναγνώσεις πάνω στα δεδομένα που εισάγαμε προηγουμένως. Τα δεδομένα προς ανάγνωση δίνονται έτοιμα σε αρχεία της μορφής `query_0x.txt` με $x=0,1,\dots,9$, ο κώδικας που εκτελείται ανά κόμβο βρίσκεται στο αρχείο `src/bin/query_throughput.rs` και το script για την παράλληλη είσοδο και λειτουργία των κόμβων βρίσκεται στον κατάλογο `experiments`. Όπως και πριν, καταγράφουμε τους χρόνους εκτέλεσης ανά κόμβο, παίρνουμε τον μέσο όρο και υπολογίζουμε το συνολικό πλήθος αναγνώσεων δια τον χρόνο αυτό. Τα αποτελέσματα φαίνονται παρακάτω:

	k=1	k=3	k=5
t=0	2987.9	2244.3	1476.1
t=1	2917.6	3905.5	5333.3

Table 2: Ρυθμαπόδοση αναγνώσεων για διαφορετικούς συνδυασμούς συνέπειας και replication factor (πράξεις ανά δευτερόλεπτο)

Για την σειριοποιήσιμη συνέπεια η αύξηση του παράγοντα αντιγραφής χειροτερεύει την απόδοση, όπως και θα περιμέναμε. Πράγματι, στο chain replication πρέπει να βρούμε τον κόμβο που είναι υπεύθυνος για αυτό το κλειδί, να προχωρήσουμε k-1 θέσεις και από εκεί να διαβάσουμε την τιμή. Η επιβάρυνση αυτή γίνεται δυσμενέστερη καθώς το k μεγαλώνει. Αντιθέτως, η αύξηση του replication factor φαίνεται να ευνοεί το eventual consistency. Πράγματι, αν το κλειδί υπάρχει στον τρέχον κόμβο, η query το επιστρέφει απευθείας. Έτσι, σε eventual consistency όχι μόνο δεν χρειάζεται επιπλέον επικοινωνία με άλλες ρέπλικες για να ληφθεί το αποτέλεσμα, αλλά για μεγάλα replication factors αυξάνεται η πιθανότητα ένα αντίγραφο του κλειδιού να βρίσκεται τοπικά. Συνεπώς, η απάντηση σε εγγραφές είναι κατά μέσο όρο γρηγορότερη και αυτό επιταχύνει το throughput του συστήματος.

Εν τέλει διαπιστώνουμε ότι παρόλο που η αυστηρότητα του linearizability εξασφαλίζει συνεπή εικόνα σε όλες τις ρέπλικες, εισάγει μεγάλη χρονική καθυστέρηση. Αντιθέτως, οι χαλαροί περιορισμοί του eventual consistency του επιτρέπουν να είναι πιο αποκρίσιμο σε πολλά αιτήματα και να κλιμακώνει καλύτερα για πολλά αντίγραφα. Η επιλογή του ενός είδους συνέπειας ή του άλλου εξαρτάται από τις απαιτήσεις της εφαρμογής, καθώς αυτή καθορίζει εάν πρέπει να δοθεί έμφαση στην συνέπεια ή στην ρυθμαπόδοση.

2.3 Πείραμα 3: Freshness & Consistency (Mixed Requests)

Στο πείραμα 3 ζητείται εκτέλεση 500 requests (queries και inserts) για k=3 και τα 2 είδη consistency με σκοπό να παρατηρήσουμε τι συμβαίνει σχετικά με το freshness των δεδομένων.

Αναλύοντας τα logs που παραχθηκαν, διαπιστώσαμε ότι, κατά την εκτέλεση με Eventual Consistency, οι ασυνέπειες που καταγράφηκαν ήταν σχετικά σπάνιες σε σχέση με τον συνολικό όγκο των αιτημάτων. Αυτό το φαινόμενο δικαιολογείται πλήρως από το γεγονός ότι η εφαρμογή εκτελέστηκε τοπικά (localhost) και όχι σε ανεξάρτητες μηχανές. Σε ένα τοπικό περιβάλλον, απουσιάζουν τα network delays και η επικοινωνία είναι αχαιαία. Έτσι, ο χρόνος που μεσολαβεί από την εγγραφή σε έναν primary node μέχρι τη διάδοση (lazy propagation) της πληροφορίας στους replica managers είναι ελάχιστος.

Παρόλα αυτά, το φαινόμενο των stale data δεν εξαλείφεται πλήρως. Όπως παρατηρήθηκε σε συγκεκριμένες περιπτώσεις στα logs, ένα Insert σε έναν κόμβο ακολουθήθηκε αμέσως από ένα Query για το ίδιο κλειδί, το οποίο όμως επέστρεψε ότι το κλειδί δεν υπάρχει. Αυτό επιβεβαιώνει τη θεωρία: το Query εξυπηρετήθηκε από τον τελευταίο replica manager, όπου η πληροφορία δεν είχε προλάβει να φτάσει.

Αντίθετα, στην περίπτωση του Chain Replication, δεν παρατηρήθηκε καμία απολύτως ασυνέπεια. Το σύστημα διατήρησε αυστηρό linearizability, διασφαλίζοντας ότι κάθε Query επέστρεφε πάντα την πιο πρόσφατη τιμή (fresh value) που είχε εισαχθεί.

Εάν η ίδια πειραματική διαδικασία εκτελούνταν σε πραγματικές υποδομές cloud, η συμπεριφορά του συστήματος θα ήταν αισθητά διαφορετική λόγω των φυσικών αποστάσεων, του

packet loss και του network latency:

- **Eventual Consistency:** Το χρονικό παράθυρο κατά το οποίο τα αντίγραφα είναι ασυγχρόνιστα θα μεγάλωνε εκθετικά. Η διάδοση των Insert και FwDelete μηνυμάτων μέσω δικτύου θα καθυστερούσε, με αποτέλεσμα τα stale data να εμφανίζονται πολύ πιο συχνά. Οι χρήστες θα διάβαζαν παλιές τιμές για σημαντικό χρονικό διάστημα μετά από ένα επιτυχές Insert.
- **Chain Replication:** Ενώ θα εξακολουθούσε να εγγυάται το linearizability (προσφέροντας τέλειο data freshness), το κόστος σε απόδοση θα ήταν τεράστιο. Το write latency θα ισούταν με το άθροισμα των Round-Trip Times (RTTs) όλης της αλυσίδας (από τον head μέχρι τον tail και πίσω), ρίχνοντας κατακόρυφα το write throughput. Επιπλέον, τα queries θα περίμεναν πολύ περισσότερο χρόνο για να αποδεσμευτούν, αυξάνοντας δραματικά τον χρόνο απόκρισης στον client.